An introduction to queuing theory

Quoting Wikipedia on the queueing theory, Little's Law states: "The long-term average number of customers in a stable system L is equal to the long-term average arrival rate, λ, multiplied by the long-term average time a customer spends in the system, W, or: L = λ W. Here:

- L = queue length, i.e., the average number of requests waiting in the system
- A = arrival rate, which is the rate at which requests enter a system
- W = wait time, which is the average time to satisfy a request

At a constant arrival rate (λ), if the queue length (L) increases, so will the wait time (W), and if the queue length (L) decreases, so will the wait time (W). So at a constant (λ), L is directly proportional to W.

Wait time (W) consists of queue time (time waiting for a resource to become available, Q) and service time (time for a resource to process a request, S). So we can say, W = Q + S. So Little's Law can be stated as: L = λ (Q + S)

The Utilisation Law is a derivative of Little's Law. It simply states that: U = (service time) x (arrival rate). For a saturated resource U is 1. This means, if the value of U reaches near 1 (or becomes 1), the resource cannot take any more requests.

Now let's check the utilisation of my server hard disk. Run *iostat -x -d /dev/sda 1* to get the values of the average reads per second, the average writes per second and the average service time. I get the following output:

```
Device: rrqm/s wrqm/s r/s w/s rsec/s    wsec/s avgrq-sz avgqu-
sz  await  svctm  %util
sda      48.00  0.00  86.00  0.00  17760.00  0.00  206.51     0.27
3.12    2.84   24.40
```

From this, I get:

- Average read request (*avgrq-sz*) = 206.51
- Average write request (*wsec/s*) = 0
- Average service time (*svctm*) = 2.84 ms

Now we know that U = S x A (can be remembered as USA, :-)). Hence, U = [(206.51 + 0) x (2.84)] / 1000 = 0.586. So my utilisation is just half way to the saturation level of 1. If this number gets closer to 1 then it's time to change the disk.

We can also find out how many requests per second this hard disk can take. Just make U=1 and find out the A (arrival rate). That is, if 1 = 2.84ms x A, then A = 1000/2.84 = 352.112 requests per second. So this hard disk can take a maximum of 353 requests per second. Anything over this number can cause the machine to fall over.

Finding hot spots in the code

We can use the *strace* tool to trace system calls and signals. *strace* runs a specified command until it exits or ends. It intercepts and records the system calls made by the process and the signals that are received by the process.

For example, let's find out the system calls made by the *who* command:

```
# strace -c who
root   pts/1      2009-09-13 10:37   (172.24.0.123)
alok   :0       2009-09-13 10:38
alok   pts/0      2009-09-13 10:38
alok   pts/1      2009-09-14 00:16
% time    seconds  usecs/call    calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
  nan   0.000000        0       20         read
  nan   0.000000        0        4         write
  nan   0.000000        0       20        1 open
  nan   0.000000        0       21         close
  nan   0.000000        0        1         execve
  nan   0.000000        0        1         time
  nan   0.000000        0       45         alarm
  nan   0.000000        0        2        2 access
  nan   0.000000        0        4         kill
  nan   0.000000        0        3         brk
  nan   0.000000        0        4         munmap
  nan   0.000000        0        3         mprotect
  nan   0.000000        0        2         _llseek
  nan   0.000000        0       30         rt_sigaction
  nan   0.000000        0       22         mmap2
  nan   0.000000        0        7        1 stat64
  nan   0.000000        0       20         fstat64
  nan   0.000000        0       31         fcntl64
  nan   0.000000        0        1         set_thread_area
------ ----------- ----------- --------- --------- ----------------
100.00   0.000000              241       4 total
```

As you can see, it clearly tells you the name of the system call and how many times it had been called.

Here I conclude for this month. Next time we will learn how to configure a tuneable RAID, using the external journal, and learn about processor scheduling using a locality of reference. END

References and attribution

- RHEL 5 IO Tuning Guide: *http://www.redhat.com/docs/wp/performancetuning/iotuning/index.html*
- Linux I/O Scheduler [WlugWiki]: *www.wlug.org.nz/LinuxIoScheduler*

By: Alok Srivastava

The author is the founder of Network NUTS and holds MCP, MCSE, MCDBA, MCT, CCNA, CCNP, RHCE and RHCSS certifications. Under his leadership, Network NUTS has been a winner of the "Best Red Hat Training Partner in North India" for the last three years in a row. He has also been a trainer for the Indian Air Force, NIC, LIC, IFFCO, Wartsila India Ltd, the government of Rajasthan, Bajaj Allianz, etc. You can reach him at *alok at networknuts dot net*.